Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique.The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size.If no suitable block is available, the process is marked as not allocated.The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

- For each process, display whether it is allocated or not.

```c
#include <stdio.h>

int main() {
    int m, n;

    // Input for memory blocks
    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);

    // Using Variable Length Arrays (C99 standard)
    int blockSize[m];
    int originalBlockSize[m];

    printf("Enter size of each block:\n");
    for (int i = 0; i < m; i++) {
        scanf("%d", &blockSize[i]);
        originalBlockSize[i] = blockSize[i]; // Keep original sizes for the final output
    }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

Best Fit Contiguous Memory Allocation

Completed

Write a C program to implement the Best Fit Contiguous Memory Allocation technique.The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size.If no suitable block is available, the process is marked as not allocated.The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

. The first line contains an integer representing the number of memory blocks.
. The second line contains the sizes of the memory blocks.
. The third line contains an integer representing the number of processes.
. The fourth line contains the sizes of the processes.

Output Format

For each process, display whether it is allocated or not.

Select Language : C (GCC 9.2.0)

```c
    // Input for processes
    printf("Enter number of processes:\n");
    scanf("%d", &n);

    int processSize[n];

    printf("Enter size of each process:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &processSize[i]);
    }

    // Best Fit Allocation Logic
    for (int i = 0; i < n; i++) {
        int best_idx = -1; // Initialize to -1 to track if we found a fit

        for (int j = 0; j < m; j++) {
            // Check if block can fit the process
            if (blockSize[j] >= processSize[i]) {
                // If it's the first block that fits, or if it's a tighter fit than the current best
```

Test Cases 2

Run Sample Cases

Run All

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

Completed

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique.The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size.If no suitable block is available, the process is marked as not allocated.The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

The first line contains an integer representing the number of memory blocks.
The second line contains the sizes of the memory blocks.
The third line contains an integer representing the number of processes.
The fourth line contains the sizes of the processes.

Output Format

For each process, display whether it is allocated or not.

Select Language : C (GCC 9.2.0)

```c
            // If it's the first block that fits, or if it's a tighter fit than the current best
            if (best_idx == -1 || blockSize[j] < blockSize[best_idx]) {
                best_idx = j;
            }
        }
    }

    // If a suitable block was found
    if (best_idx != -1) {
        // Deduct the process size to update the remaining space in the block
        blockSize[best_idx] -= processSize[i];

        // The fragment is now just the updated remaining size of the block
        int fragment = blockSize[best_idx];

        printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
               i + 1, processSize[i], best_idx + 1, originalBlockSize[best_idx], fragment);
    } else {
        // If best_idx is still -1, no block was large enough
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

Completed

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique.The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size.If no suitable block is available, the process is marked as not allocated.The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

. The first line contains an integer representing the number of memory blocks.
. The second line contains the sizes of the memory blocks.
. The third line contains an integer representing the number of processes.
The fourth line contains the sizes of the processes.

Output Format

Select Language : C (GCC 9.2.0)

```c
        // If a suitable block was found
        if (best_idx != -1) {
            // Deduct the process size to update the remaining space in the block
            blockSize[best_idx] -= processSize[i];

            // The fragment is now just the updated remaining size of the block
            int fragment = blockSize[best_idx];

            printf("Process %d of size %d is allocated to Block %d of size %d with Fragment
                   i + 1, processSize[i], best_idx + 1, originalBlockSize[best_idx], fragme
        } else {
            // If best_idx is still -1, no block was large enough
            printf("Process %d of size %d is not allocated\n", i + 1, processSize[i]);
        }
    }

    return 0;
}
```

Test Cases 2

Run Sample

Custom Test Case

> Custom Test